

HRM - Human Resource Management Web App

Project	HRM - Human Resource Management Web App
Frontend	React 19, Vite 7, JavaScript, CSS, Lucide React icons
Backend	ASP.NET Core Web API, .NET 8, Entity Framework Core 8, JWT Authentication
Database	Microsoft SQL Server, managed locally through SQL Server Management Studio 22
Local URL	Frontend: http://127.0.0.1:5174/ Backend: https://localhost:7237

Project Description

HRM is a single-page Human Resource Management web application built for Assignment 2. The website helps a small company manage its core HR records in one place, rather than keeping employee, department, transfer, salary, and payroll information in separate spreadsheets.

The system supports secure account-based access. After a user signs up and logs in, the business units, departments, designations, employees, salary structures, salary assignments, adjustments, and payroll runs are associated with that logged-in account. This keeps each user's HR workspace separate.

The main problem solved by this project is organising company HR operations through a browser-based interface with database-backed CRUD operations, authentication, search, and payroll calculation workflows.

Main Features

- User signup and login with password hashing and JWT authentication.
- User-specific data ownership for HR records.
- Set up CRUD for business units, departments, designations, and employment types.
- Employee CRUD with a searchable employee directory.
- Employee transfer and promotion workflow between departments and designations.
- Transfer/promotion history tracking.
- Salary structure creation with editable salary components and percentages.
- Salary assignment to employees, including update and delete options.

- Monthly salary adjustments for one employee, all employees in one business unit, or all employees in all business units.
- Payroll run screen to generate monthly salary totals using assigned salary and adjustments.
- Search/filter support in the major listing screens.
- React single-page application behaviour with no full-page reload during normal use.

Technical Stack

Frontend

- React 19
- Vite 7
- JavaScript
- CSS
- Lucide React icons

Backend

- ASP.NET Core Web API
- .NET 8
- Entity Framework Core 8
- JWT Bearer Authentication
- Swagger/OpenAPI for API testing

Database

- Microsoft SQL Server
- SQL Server Management Studio 22 can be used to inspect and manage the database.
- Default local database name: HRM

Folder Structure

Root folder

- **HRM.sln** – Visual Studio solution file that contains and manages the backend HRM project.
- **README.md** – Project documentation file that may include setup instructions, project overview, and usage details.
- **HRM/** – Main ASP.NET Core backend project folder. Contains controllers, services, models, DTOs,

database context, migrations, and backend configuration files.

- **hrm-client/** – React frontend project folder. Contains the Vite React app, frontend source code, styling, and frontend configuration files.

HRM/

- **HRM.csproj** – ASP.NET Core project configuration file. Defines backend dependencies, target framework, and project settings.
- **Program.cs** – Backend startup file. Configures dependency injection, CORS, Swagger, authentication, authorisation, and application middleware.
- **appsettings.json** – Application configuration file. Stores local database connection string, JWT settings, and other backend configuration values.

Controllers/

- **AuthController.cs** – API endpoints for user authentication, login, registration, and JWT-based access.
- **BasicController.cs** – API endpoints for setup/basic data such as departments, designations, business units, and employment types.
- **EmployeeController.cs** – API endpoints for employee information, employee records, and employee management operations.
- **PayrollController.cs** – API endpoints for payroll-related operations and payroll processing.
- **SalaryController.cs** – API endpoints for salary setup, salary records, and salary management.
- **BonusController.cs** – API endpoints for employee bonus-related operations.
- **PdfReportController.cs** – API endpoints for generating and downloading PDF reports.

Data/

- **HRMContext.cs** – Entity Framework Core database context. Defines database tables using DbSet properties and manages communication between the backend and SQL Server database.

DTOs/

- **AuthDTO.cs** – Data transfer objects used for authentication requests and responses.
- **EmployeeDTO.cs** – Data transfer objects used for employee-related data between frontend and backend.
- **PayrollDTO.cs** – Data transfer objects used for payroll-related requests and responses.
- **SalaryDTO.cs** – Data transfer objects used for salary-related operations.

Interfaces/

- **Interfaces/** – Contains service contracts used by the backend. These interfaces define the structure of backend services and help keep the application organised and maintainable.

Migrations/

- **Migrations/** – Contains Entity Framework Core migration files. These files are used to create, update, and maintain the SQL Server database structure.

Models/

- **AppUser.cs** – Entity model for application users and authentication-related user data.
- **BusinessUnit.cs** – Entity model for business unit information.
- **Department.cs** – Entity model for department records.
- **Designation.cs** – Entity model for employee designation records.
- **EmpBasicInfo.cs** – Entity model for storing basic employee information.
- **EmpTransferNPromotion.cs** – Entity model for employee transfer and promotion history.
- **EmploymentType.cs** – Entity model for employment type information.
- **Payroll/** – Contains payroll-related entity models.
- **Salary/** – Contains salary-related entity models.
- **Bonus/** – Contains bonus-related entity models.

Services/

- **AuthService.cs** – Business logic for user authentication, registration, login, and JWT token handling.
- **BasicService.cs** – Business logic for managing setup data such as departments, designations, business units, and employment types.
- **CurrentUserService.cs** – Handles current logged-in user information and user ownership-related logic.
- **EmployeeService.cs** – Business logic for employee management, employee records, and employee-related operations.
- **PayrollService.cs** – Business logic for payroll processing and payroll record management.
- **SalaryService.cs** – Business logic for salary setup, salary records, and salary-related operations.

hrm-client/

- **index.html** – Main HTML entry point for the React single-page application.
- **package.json / package-lock.json** – React project configuration and dependency lock files. Defines frontend dependencies, scripts, and project settings.
- **vite.config.js** – Vite configuration file. Includes frontend build settings and backend API proxy configuration.

hrm-client/src/

- **main.jsx** – Main React application file. Contains frontend state management, screens, forms, API calls, and UI logic.
- **styles.css** – Styling file for the React application, including layout, forms, tables, buttons, and responsive design.
- **assets/** – Contains frontend assets such as images, icons, logos, and other static files used in the React app.

How To Run The App

Prerequisites

Install these first:

- .NET 8 SDK
- Node.js and npm
- SQL Server Express or SQL Server Developer Edition
- SQL Server Management Studio 22

1. Configure The Database Connection

Open:

```
HRM/appsettings.json
```

Update the connection string if your SQL Server name is different:

```
"DefaultConnection":
```

```
"Server=localhost\\SQLEXPRESS;Database=HRM;Trusted_Connection=True;Encrypt=False;TrustServerCertificate=True"
```

For example, change only the Server= value if your server name is different in SSMS.

2. Apply Database Migrations

From the project root, run:

```
dotnet ef database update --project HRM/HRM.csproj
```

If dotnet ef is not installed, install it first:

```
dotnet tool install --global dotnet-ef
```

3. Start The Backend API

From the project root, run:

```
dotnet run --project HRM/HRM.csproj --launch-profile HRM
```

The backend runs on:

```
https://localhost:7237
```

```
http://localhost:5118
```

4. Start The Frontend

Open a new terminal and run:

```
cd hrm-client  
npm install  
npm start
```

The frontend runs on:

```
http://127.0.0.1:5174/
```

5. First Use Workflow

After opening the app:

1. Create an account from the signup screen.
2. Log in with the account.
3. Add setup data: business units, departments, designations, and employment types.
4. Create employees.
5. Use Transfer for department/designation changes.
6. Create salary structures.
7. Assign salary structures to employees.
8. Add monthly adjustments if required.
9. Generate payroll for the selected month from the Payroll screen.

Workload Allocation

Md Asadul Hoque Chowdhury (25621350): Done the whole project solely (Backend + Frontend)